Data Structures and Algorithms

LECTURE 03: RECURSION

greenwich.edu.vn

UNIVERSITY OF GREENWICH

Alliance with FPT Education

Contents

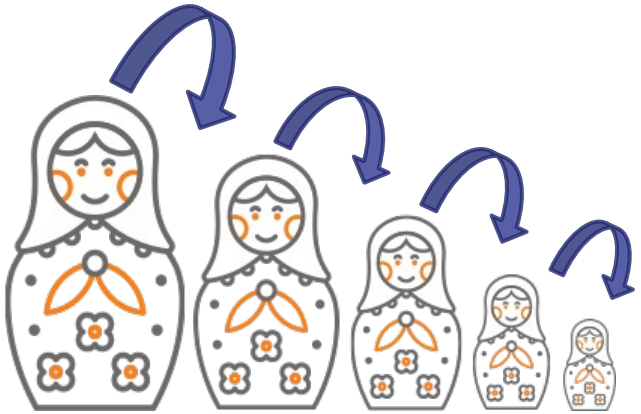
- Recursion
- Generating Simple Combinations
- Recursion or Iteration?
  - Harmful Recursion and Optimizing Bad Recursion

Recursion

- What is Recursion?
  - **Method** of solving a problem where the solution depends on solutions to smaller instances of the same problem
  - A common **computer programing tactic** is to **divide** a problem into **sub-problems** of the same type as the original, **solve** those sub-problems, and **combine** the **results**.

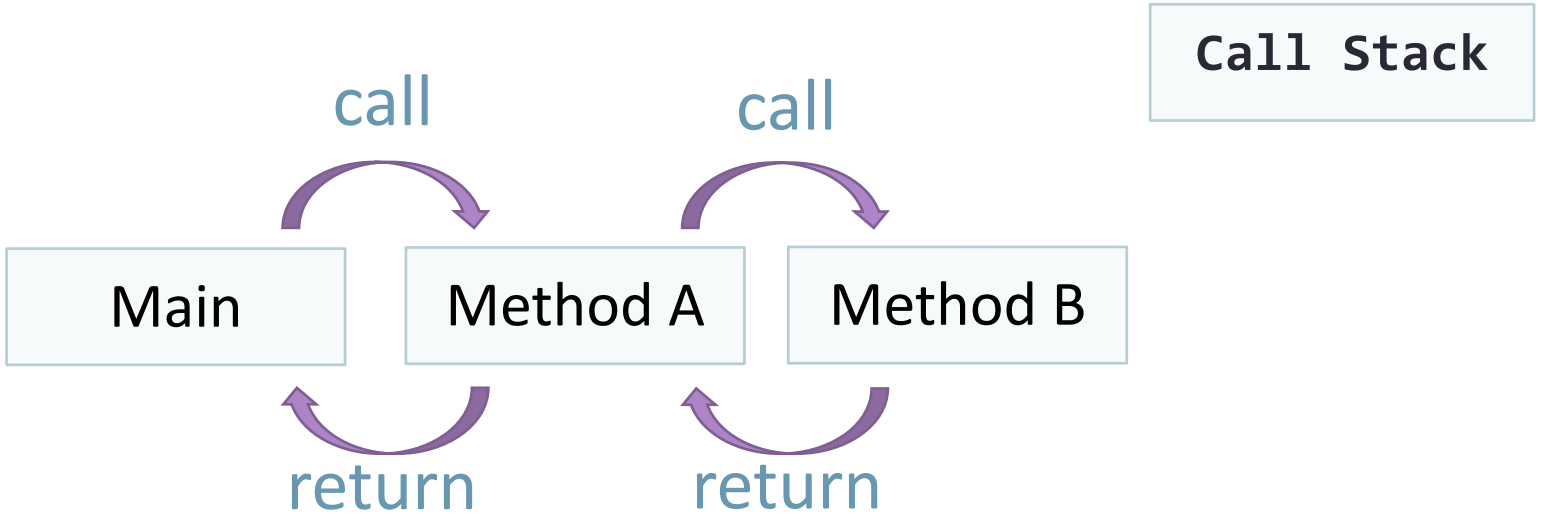
Recursion

- What is Recursion?
  - A function or a method that **calls itself one or more** times until a specified **condition** is **met**
  - After the recursive call the rest code is processed **from** the **last** one called **to** the **first**.



Call Stack

- "The stack" is a small **fixed-size** chunk of memory (e.g. 1MB)
- Keeps track of **the point** to which each active subroutine should **return control** when it **finishes**



Other Definition

- Problem solving technique (In CS)
  - Involves a **function calling itself**
  - The function should have a **base case**
  - **Each step** of the recursion should **move towards** the **base**

sum(array)

1	2	3	4	5

array[0] + sum(sub-array)

1	+	2	3	4	5

Array Sum – Example

Sum(n)

1	2	3	4

Sum(n - 1)

1 + 2 3 4

Sum((n – 1) - 1)

1 + 2 + 3 4

Sum(((n – 1) - 1) – 1)

1 + 2 + 3 + 4

Problem: Array Sum

- Create a **recursive method** that
  - Finds the sum of all numbers stored in an
  - Read numbers from the console

```
1 2 3 4
```

➡

```
10
```

```
-1 0 1
```

➡

```
0
```

Solution: Array Sum

- Sample source code

```
static int sum(int[] array, int index) {
    if (index == array.length() - 1) {
        return array[index];
    }

    return array[index] + sum(array, index + 1);
}
```

Base case

Recursive call

Problem: Recursive Factorial

- Create a **recursive method** that calculates **n!**
  - Read n from the console

`5`	➡	`120`
`10`	➡	`3628800`

Recursive Factorial – Example

- Recursive definition of **n!** (n factorial):

```
n! = n * (n-1)! for n > 0
0! = 1
```

3!	=	3 * 2!
2!	=	2 * 1!
1!	=	1 * 0!
0!	=	1

Solution: Recursive Factorial

- Sample source code

```
static long factorial(int num) {
    if (num == 0) {
        return 1;
    }

    return num * factorial(num - 1);
}
```

Base case

Recursive call

Direct and Indirect Recursion

- Direct recursion
  - A method directly calls itself
- Indirect recursion
  - Method **A** calls **B**, method **B** calls **A**
  - Or even **A → B → C → A**

Recursion Pre-Actions and Post-Actions

- Recursive methods have **three** parts:
  - **Pre-actions** (before calling the recursion)
  - **Recursive calls** (step-in)
  - **Post-actions** (after returning from recursion)

```
static void recursion() {
  // Pre-actions

  recursion();

  // Post-actions
}
```

Problem: Recursive Drawing

- Create a **recursive method** that draws the following figure

5

```
*****
****
***
**
*
#
##
###
####
#####
```

Pre-Actions and Post-Actions – Example

- Sample source code

```
static void printFigure(int n) {
    if (n == 0) {
      return;
    }
    // TODO: Pre-action: print n asterisks
    printFigure(n - 1);
    // TODO: Post-action: print n hashtags
}
```

Base case

Recursive call

Recursive Algorithm

- Generating Simple Combinations

Generating 0/1 Vectors

- How to generate all 8-bit vectors **recursively**?

```
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 1
...
0 1 1 1 1 1 1 1
1 0 0 0 0 0 0 0
...
1 1 1 1 1 1 1 0
1 1 1 1 1 1 1 1
```

Generating 0/1 Vectors

- Start with a **blank vector**

0	…	…	…	…	…	…	…

- Choose the **first position** and **loop through all possibilities**

0	…	…	…	…	…	…	…

$\underbrace{\qquad\qquad\qquad\qquad}_{n\ -\ 1}$

1	…	…	…	…	…	…	…

$\underbrace{\qquad\qquad\qquad\qquad}_{n\ -\ 1}$

- For each possibility, generate all **(n – 1)-bit** vectors

Solution: Generate n-bit Vectors

```
static void gen01(int index, int[] vector) {
    if (index >= vector.length()) {
        print(vector);
    } else {
        for (int i = 0; i <= 1; i++) {
            vector[index] = i;
            gen01(index + 1, vector);
        }
    }
}
```

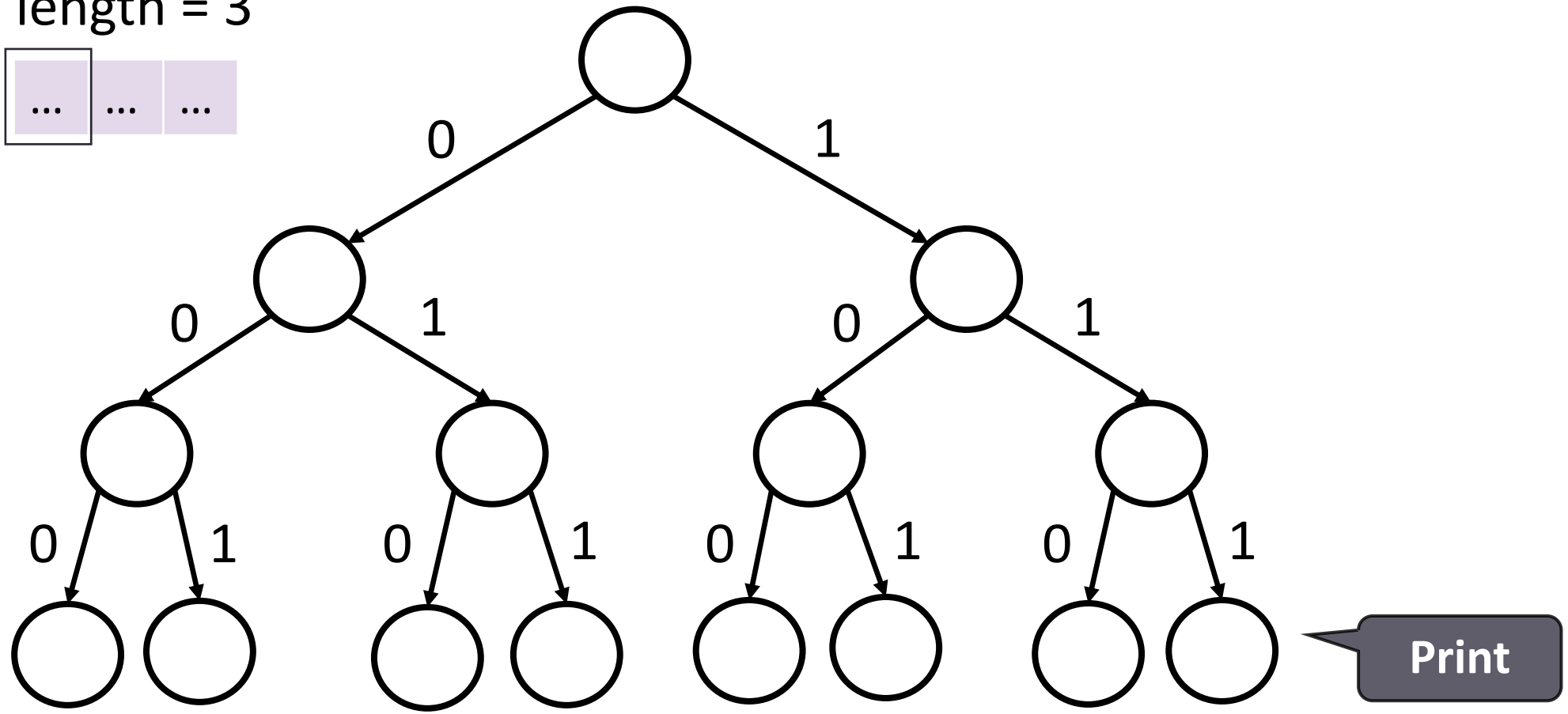
Base case

Recursive call

Generating 3-bit Vectors Recursion Tree

length = 3

vector: | … | … | … |



Performance: Recursion vs. Iteration

- Recursive calls are **slightly slower**
- Parameters and return values **travel** through the stack
- Good for branching problems

```
static long recurFact(int n) {
  if (n == 0)
    return 1;

  return n * Fact(n - 1);
}
```

- No function call **cost**
- Creates **local** variables
- Good for linear problems (no branching)

```
static long iterFact(int num) {
  long result = 1;
  for (int i = 1; i <= n; i++)
    result *= i;
  return result;
}
```

Infinite Recursion

- **Infinite recursion** == a method calls itself infinitely
  - Typically, infinite recursion == bug in the program
  - The bottom of the recursion is missing or wrong
  - In C# / Java / C++ causes "stack overflow" error

```
recurr:10, Main (recursionLab)
recurr:12, Main (recursionLab)
recurr:12, Main (recursionLab)
recurr:12, Main (recursionLab)
recurr:12, Main (recursionLab)
recurr:12, Main (recursionLab)
main:5, Main (recursionLab)
```

```
Exception in thread "main" java.lang.StackOverflowError
    at recursionLab.Main.recurr(Main.java:9)
    at recursionLab.Main.recurr(Main.java:9)
    at recursionLab.Main.recurr(Main.java:9)
    at recursionLab.Main.recurr(Main.java:9)
    at recursionLab.Main.recurr(Main.java:9)
    at recursionLab.Main.recurr(Main.java:9)
    at recursionLab.Main.recurr(Main.java:9)
```

Recursion Can be Harmful!

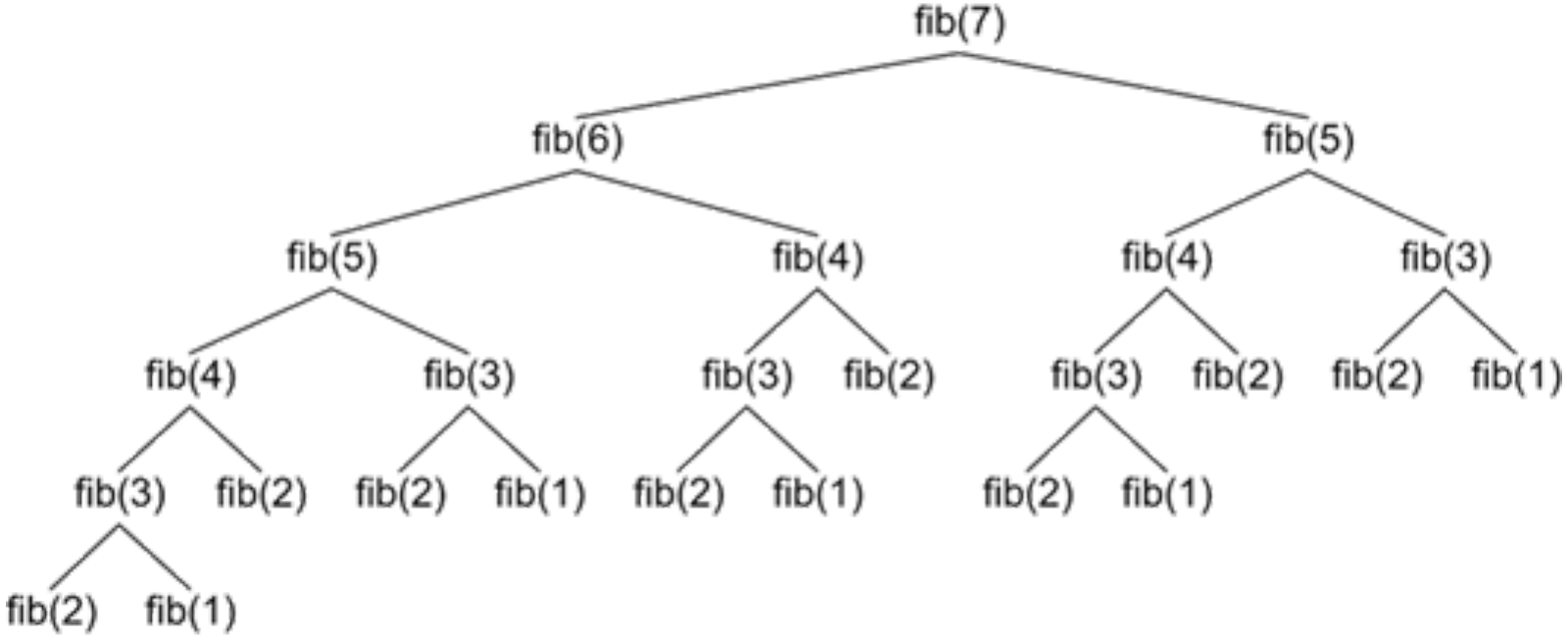
- When used incorrectly recursion could take too much memory and computing power

```
static long fibonacci(int n) {
    if (n <= 1)
        return 1;
    return fibonacci(n - 1) + fibonacci(n - 2);
}

public static void main(String[] args) {
    System.out.println(fibonacci(10)); // 89
    System.out.println(fibonacci(50)); // This will hang!
}
```

How the Recursive Fibonacci Calculation Works?

- `fib(n)` makes about `fib(n)` recursive calls
- The same value is calculated many, many times!



When to Use Recursion?

- Avoid recursion when an **obvious** iterative algorithm **exists**
  - Examples: **factorial**, **fibonacci** numbers
- Use recursion for **combinatorial** algorithms where
  - At each step you need to **recursively** explore more than one possible continuation, i.e. **branched** recursive algorithms

Summary

- **Recursion**
- When **to use recursion**
- When **to use iteration**